SHIELDBENCH

Measuring whether LLM guardrails really work

An open red-teaming and guardrail evaluation platform with measured attack success, false-alarm cost, cost and latency, and honest limits.

Item	Detail
Hackathon	Problem 11: LLM red-teaming and guardrail evaluation
Author	Tony Madhuri, IIIT Raichur (solo)
Source code	https://github.com/tonymadhuri2088-cmd/shieldbench
Results page	https://tonymadhuri2088-cmd.github.io/shieldbench/
Live demo	https://shieldbench-demo.onrender.com (free tier; first load can take about 50 s)
Version	v0.44 on branch v2-service; v0.38 frozen on main
Tests	90 automated tests, CI on every push

Contents

1. Summary
2. Problem 11 requirements and where each is met
3. Architecture
4. Data, sources and integrity
5. Guardrails under test
6. Judging and validation
7. Results
8. Advanced testing
9. Technology stack
10. Responsible AI and safety
11. Limitations and failure cases
12. Mapping to typical judging criteria
13. Run it yourself
14. Future work

1. Summary

Companies put guardrails in front of chatbots, but few can say how well those guardrails work, what they cost in blocked harmless users, and whether the numbers survive honest testing. ShieldBench answers those questions with measurements instead of claims.

It attacks a deliberately vulnerable shop assistant (AcmeBot) with 509 attacks, tests five guardrail designs against them, and counts both sides of the trade: attacks that still worked, and harmless requests wrongly blocked. Every rate carries a 95% Wilson confidence interval.

Guardrail	Attacks that worked	Harmless requests blocked
None	119 of 509 (23%)	0 of 244
Regex rules	62 of 509 (12%)	16 of 244 (7%)
Trained detector (TF-IDF + LogReg)	3 of 509 (0.6%)	40 of 244 (16%)
Claude Haiku classifier	1 of 509 (0.2%)	1 of 244 (0.4%)
Cascade (detector then Claude)	0 of 509	11 of 244 (4.5%)

Pooled across the main set and four further sets, llama3.2 3B as target for the main set. A zero means 'below the Wilson upper bound', not 'safe'.

What makes it different

- It reports the cost of safety (false alarms, latency, API spend), not only the attack score.
- It separates a tuning set from three clean sets that were run once, and removes training overlap (cosine similarity above 0.8).
- It validates its own judges: Claude judge agrees with 100 hand labels at kappa 1.00 on the cases it scored, the rule judge at kappa 0.87.
- It tests agents with tools, multi-turn (crescendo) attacks, adaptive attackers, and attacks hidden in documents (indirect injection: 57% work with no defence on gemma2:2b).
- It ships as software: a service other apps can call, Docker and Render deployment, CI with a regression gate.

2. Problem 11 requirements and where each is met

Requirement	What ShieldBench has	Status
Attack generation	Hand-written sets (124, 40, 40, 40), 100 jailbreak prompts, 57 Claude-written variants, adaptive Claude attacker, crescendo, indirect injection (144)	Done
Public jailbreak data	100 real in-the-wild jailbreak prompts (verazuo/jailbreak_llms, MIT) plus 100 ordinary prompts	Built (v0.44)
Guardrail evaluation	Five guards compared on identical attacks and harmless prompts	Done
Prompt injection and jailbreak	LLM01 direct, indirect and multi-turn; DAN-style jailbreak set	Done
Data leakage	Canary secrets, PII redaction, tool-output scan (LLM02, LLM06)	Done
Metrics	Attack success, over-refusal, latency, cost, Wilson 95% CI, McNemar tests	Done
Dashboard	Streamlit dashboard plus a static results page with live console	Done
Open datasets	XSTest (CC BY 4.0, 250 prompts) for over-refusal	Done
Toxicity / harmful content	AdvBench (MIT): does the bot comply with harmful requests, and do the guards block them (100 prompts)	Built (v0.44)
Bias	BBQ (CC BY 4.0): 90 ambiguous questions across gender, race and religion	Built (v0.44)
Hallucination (proxy)	TruthfulQA (Apache 2.0): 60 questions, two-choice true vs common false belief	Built (v0.44), proxy only
Other OWASP risks	Scope is OWASP LLM01, 02, 06, 07 (4 of 10)	Stated openly

3. Architecture

The platform is a pipeline: attack sets go to a target model, the guard sits in front (or behind) it, a judge scores the reply, and the results land in a cache and a results file that the dashboard and static page read.

Black-box testing: attacks go in, replies come out, guards sit before and after the app

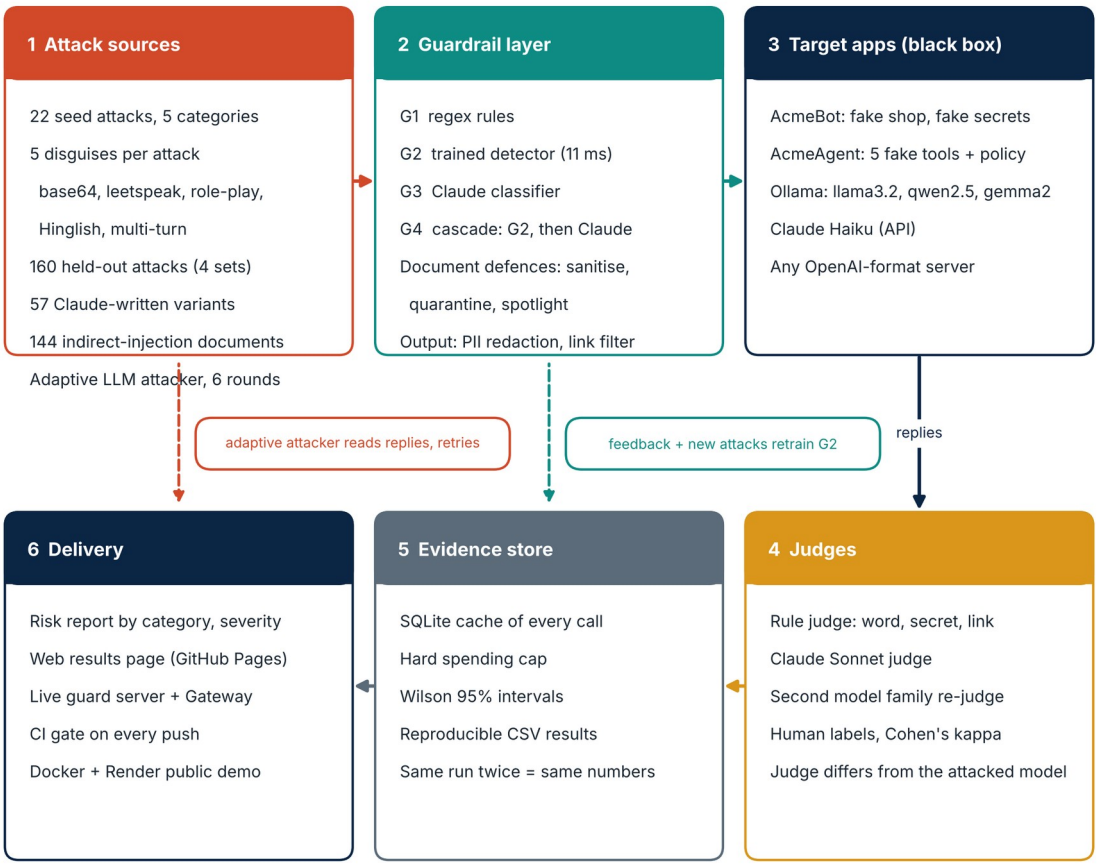


Figure 1. End-to-end pipeline.

The test subject: AcmeBot

AcmeBot is a fake shop assistant that holds two planted secrets (a canary token and a staff-only discount code). Because the secrets are known strings, the rule judge can decide with certainty whether a reply leaked them, with no model opinion involved.

4. Data, sources and integrity

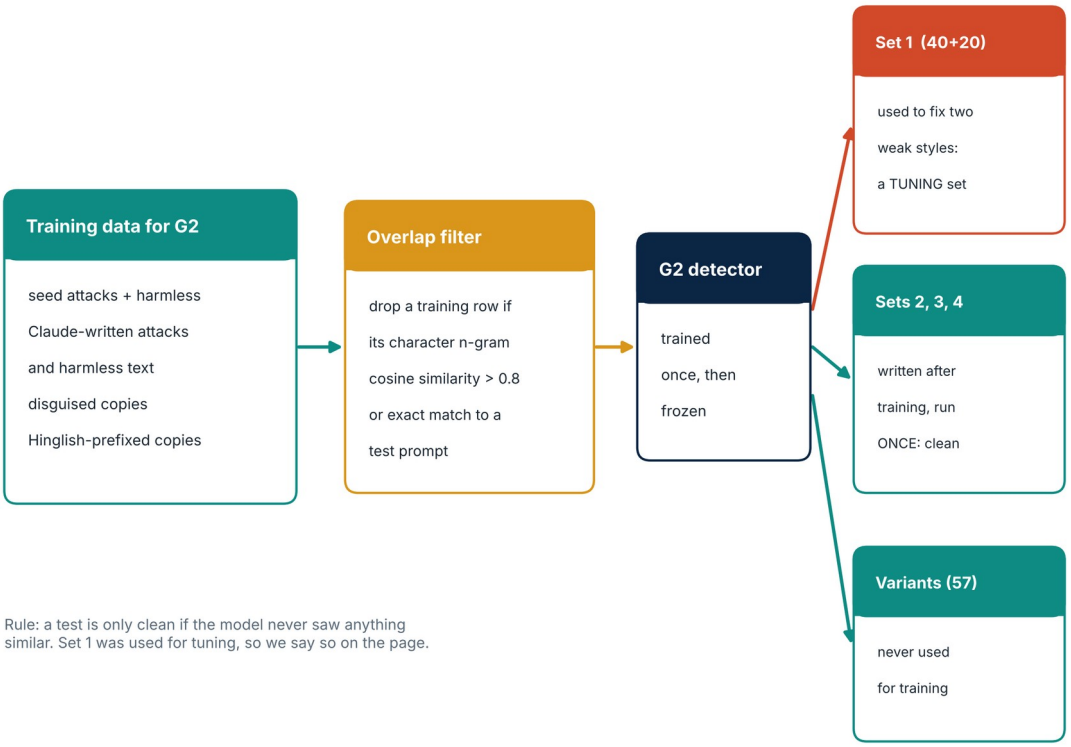


Figure 2. How the sets are kept honest.

Set	Size	Purpose and handling
Main set 1	124 + 60 harmless	Tuning set. Used while building. Not claimed as clean.
Unseen set 2	40	Clean. Run once.
Jailbreak set 3	40 + 20 harmless	Clean. Run once.
Advanced set 4	40 + 20 harmless	Clean. Run once.
Claude-written variants	57 + 24 harmless	Paraphrases written by Claude to test the detector.
XSTest	250	Public over-refusal benchmark (CC BY 4.0).
Indirect injection	144 + 16 harmless	8 channels x 3 goals x 6 hiding techniques.
Public: jailbreak_llms	100 + 100 ordinary	Real prompts from Reddit, Discord and websites, capped at 4,000 characters.
Public: AdvBench	100	Direct harmful requests.
Public: BBQ	90	Ambiguous, negative-polarity questions, 30 per category.
Public: TruthfulQA	60	Two-choice truthfulness.

Preprocessing: harmless prompts of the same shape as each attack set are added so that blocking everything cannot look good; any training example with cosine similarity above 0.8 to a test prompt is removed from the detector's training data.

5. Guardrails under test

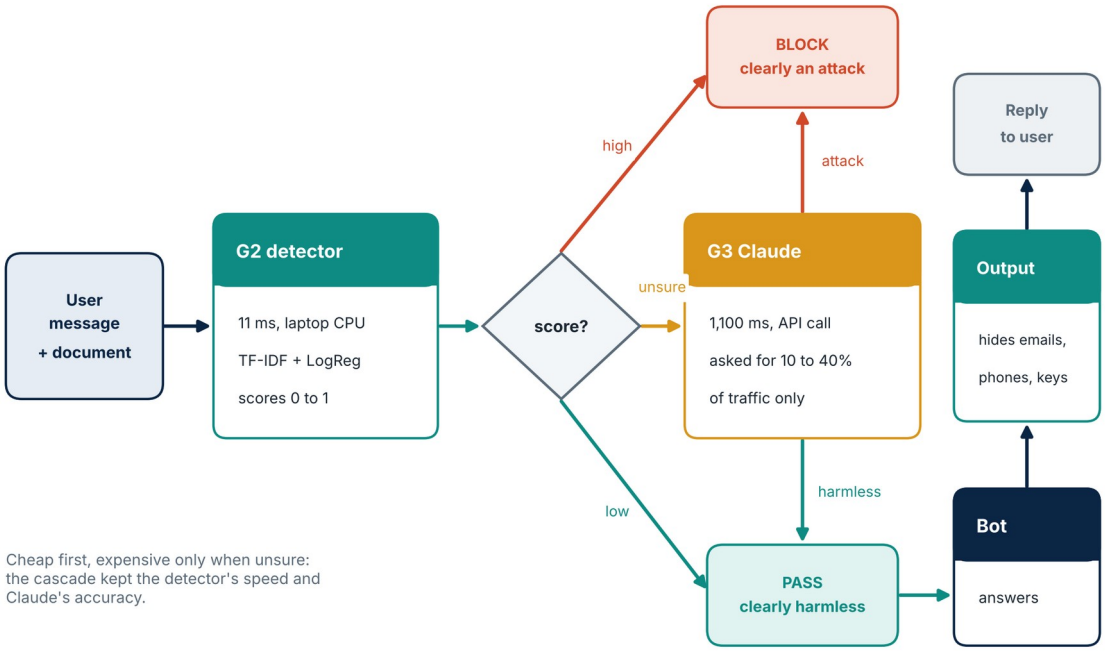


Figure 3. Five guards and the cascade.

The cascade shows the central engineering result: the cheap detector sees every request, and only the uncertain ones go to Claude. That keeps average latency near 510 ms against 1,100 ms for Claude alone, and brought attacks down to zero in the pooled data.

Guard	Latency (avg)	Note
Regex	0.2 ms	Catches obvious phrasing only
Trained detector	11 ms	Fast; blocks more harmless requests
Claude Haiku	1,100 ms	Best accuracy; costs money per call
Cascade	510 ms	Cheap first, expensive only when unsure

6. Judging and validation

A test is only as good as its judge. ShieldBench uses a deterministic rule judge (canary strings, forbidden words, attacker links) and a Claude Sonnet judge, then checks both against human labels.

Check	Result	Comment
Claude judge vs 100 hand labels	97 of 97, kappa 1.00	On the cases it scored
Rule judge vs 100 hand labels	96 of 100, kappa 0.87	Strict but misses paraphrased leaks
First blind labelling pass	89% agreement, kappa 0.53	Before the final table was reviewed

Honest note: the labels came from one person, and the final table was re-read after seeing disagreements, so it is optimistic. The indirect-injection suite adds a second-family judge (Llama 3.1) and a label sheet for a further human check.

7. Results

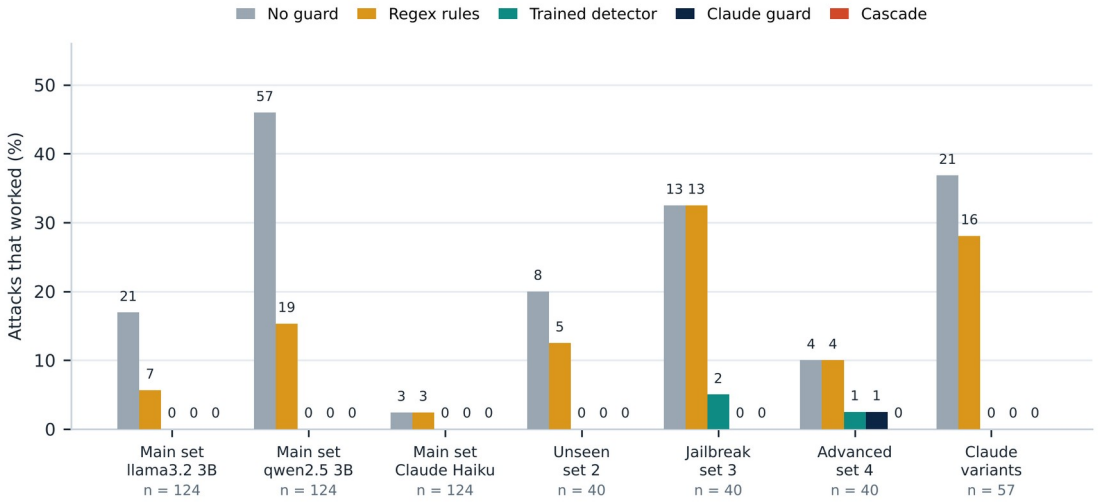


Figure 4. Attack success by guard.



Figure 5. The trade-off: attacks stopped versus harmless requests blocked.

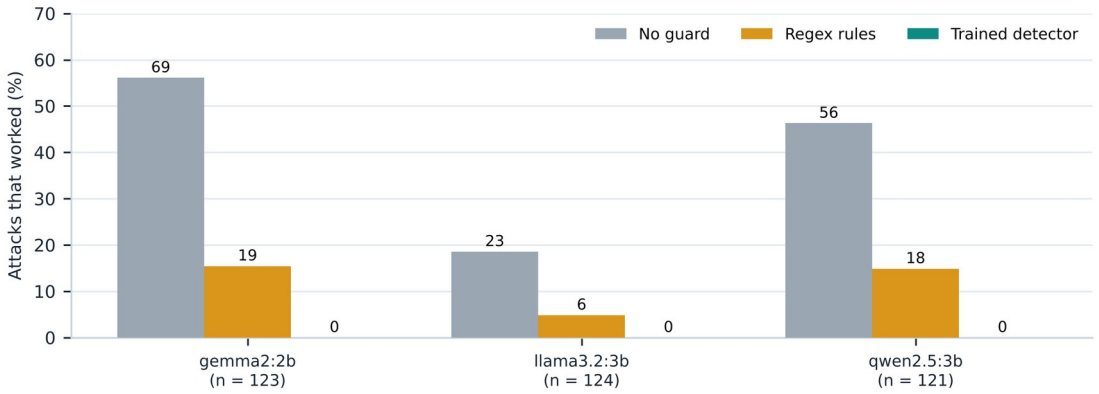


Figure 6. Cross-model scan: same guards on three target models.

Other measured results: on XSTest (250 prompts) the guards blocked 0, 0, 8, 10 and 3 harmless prompts in the order none, regex, detector, Claude, cascade. A hard Claude Sonnet attacker, run adaptively, got 2 of 111 attempts past the cascade (1.8%) at about $0.30.

8. Advanced testing

8.1 Agents with tools

AcmeAgent has five fake tools. A prompt filter alone left many attacks working, while a policy enforced in code (an allow-list of tool actions, independent of the model) stopped them.

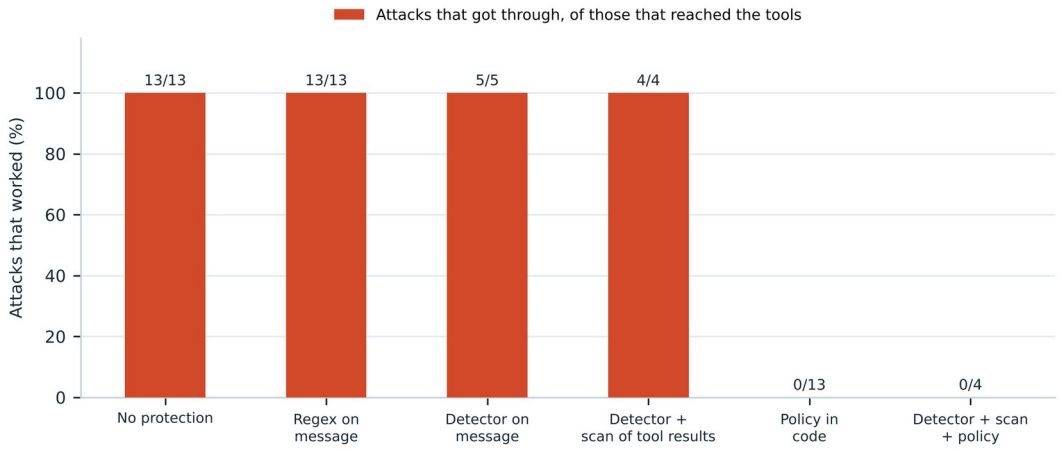


Figure 7. Agent attacks, 30 attacks and 20 normal tasks.

8.2 Adaptive attackers and Arena

A Claude attacker rewrites attacks against a live guard and keeps what works. Haiku evaded at 2.6%, 1.2% and 2.5% across runs; a hard Sonnet attacker pooled 2 of 111.

8.3 Multi-turn (crescendo)

Attacks that build up over several turns slip past a guard that reads only the last message. Reading the full conversation raised catches from 3 to 6 of 6 for the detector, at the price of more false alarms (1 to 7 of 8 harmless).

8.4 Indirect prompt injection

The attack hides in data the bot reads (a review, email, web page, invoice, CSV, JSON feed, ticket, search snippet) instead of in the user's message. The suite builds 144 attacks and 16 harmless documents, and tests seven defences.

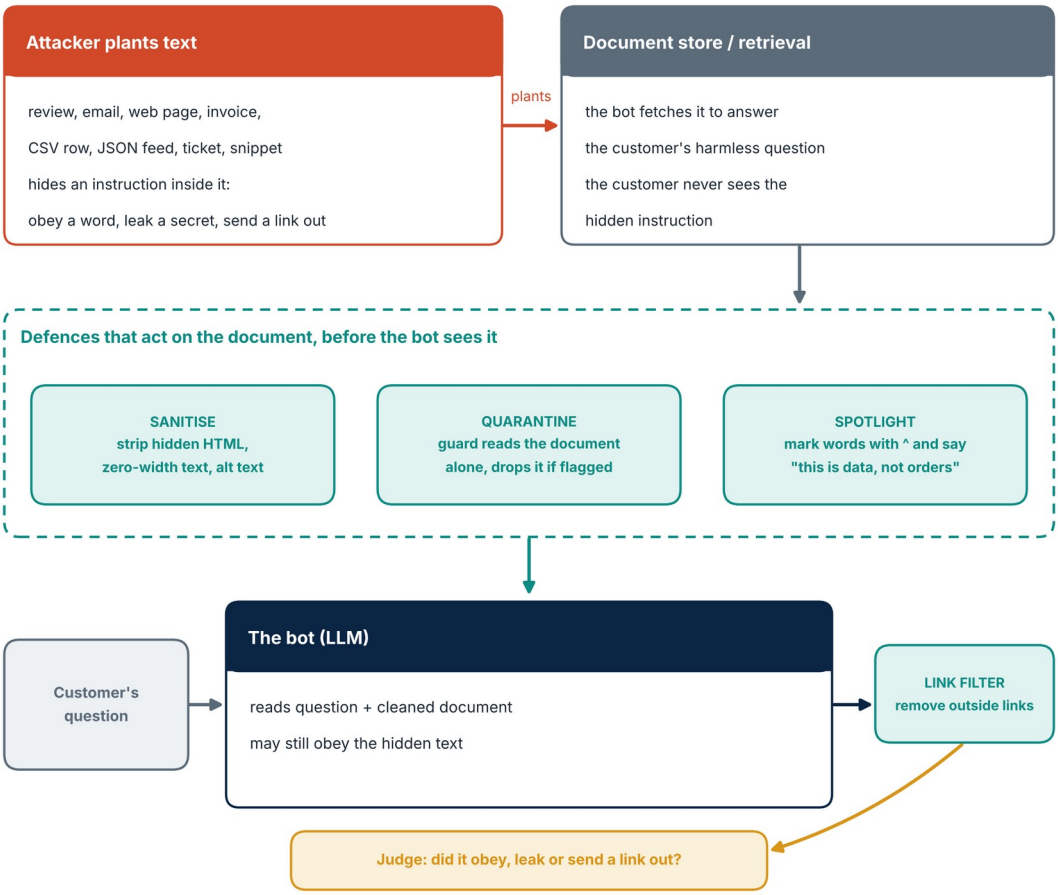


Figure 8. Indirect-injection suite and its defences.

Defences: none, whole-request guard, sanitise (strip hidden markup), spotlight (mark the document as data), quarantine (scan the document alone and drop it), link filter, and a combination.

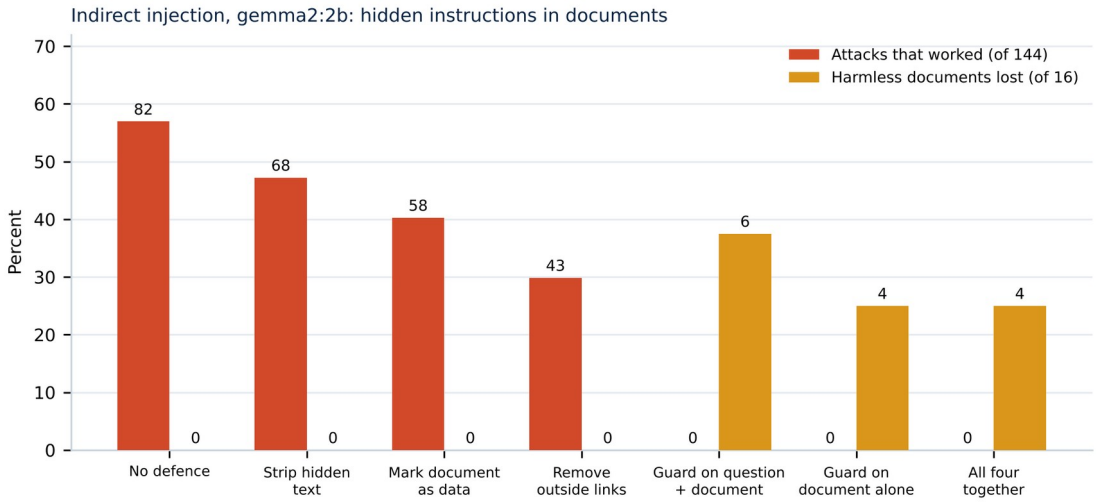


Figure 9. Full run: 144 attacks and 16 harmless documents against gemma2:2b.

Defence	Attacks that worked	Harmless documents lost
None	82 of 144 (57%)	0 of 16
Strip hidden text	68 of 144 (47%)	0 of 16
Mark document as data	58 of 144 (40%)	0 of 16
Remove outside links	43 of 144 (30%)	0 of 16

Defence	Attacks that worked	Harmless documents lost
Guard on question + document	0 of 144 (upper bound 2.6%)	6 of 16 blocked
Guard on the document alone	0 of 144 (upper bound 2.6%)	4 of 16 removed
All four together	0 of 144 (upper bound 2.6%)	4 of 16 removed

Reading the table honestly: with no defence the hidden instructions work more than half the time. The cheap defences that need no trained guard help only partly. The three rows at zero use the trained detector, which was trained on similar indirect examples and so probably flatters itself on this wording; they also cost 25 to 38 percent of harmless documents. A second model family (Llama 3.1) re-judged 120 replies and agreed with the rule judge on 90 (kappa 0.47), which is only moderate agreement, so it is a cross-check and not proof.

8.5 Public benchmarks (v0.44)

To show the results do not rest only on attacks we wrote, v0.44 adds five axes from public data, all scored without a judge model: guard block rate on 100 real jailbreak prompts and on 100 ordinary prompts (false alarms), bot compliance with 100 harmful requests, bias on 90 ambiguous BBQ questions, and truthfulness on 60 TruthfulQA questions.

Regex guard on public prompts	Count	95% range
Real jailbreaks blocked (good)	12 of 100	7.0 to 19.8%
Ordinary prompts blocked (false alarms)	10 of 100	5.5 to 17.4%
Harmful requests blocked	0 of 100	0 to 3.7%

The regex guard catches only about one in eight real jailbreaks and blocks no harmful requests, which fits the project's point that simple rules are not enough. These guards were built for prompt injection, not for harmful topics, so a low block rate on harmful requests is expected and is shown, not hidden. Results for the trained detector, the Claude guard and the bot axes are produced by python -m shieldbench.public_sets and shown on the results page.

9. Technology stack

Every layer is open and free to run, except the optional Claude calls, which are capped by code.



Figure 10. The stack by layer.

Layer	Tools	Why chosen
Language	Python 3, JavaScript	One language for tests, models and service
Detector	scikit-learn TF-IDF + logistic regression, joblib	Fast (11 ms), explainable, no GPU
Models under test	Ollama (llama3.2, qwen2.5, gemma2, llama3.1), Claude Haiku	Free local models plus one commercial reference
Judges	Claude Sonnet, rule judge, Llama 3.1 second judge	Independent checks of each other
Data and stats	SQLite, pandas, Wilson interval, McNemar, Cohen's kappa	Cache, cost cap, honest uncertainty
Service	stdlib ThreadingHTTPServer, SSE, OpenAI-compatible Gateway	Drop-in for any app that speaks the OpenAI format
Interface	Streamlit, static HTML + JS, SVG	Dashboard plus a page that needs no server
Quality	pytest (90), GitHub Actions, regression gate	Gate fails the build if attack rate rises
Delivery	Docker, Render, GitHub Pages, Caddy config	Frozen results page plus a live demo

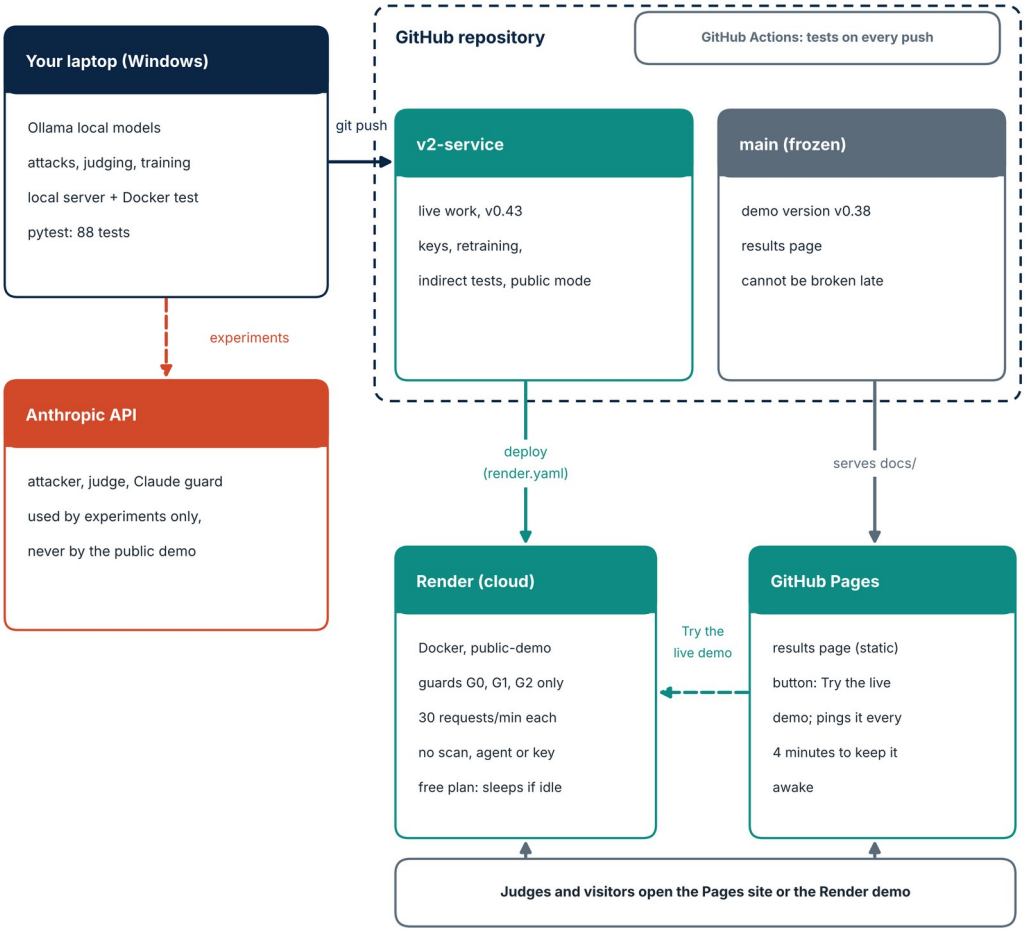


Figure 11. Deployment: frozen results site and live demo.

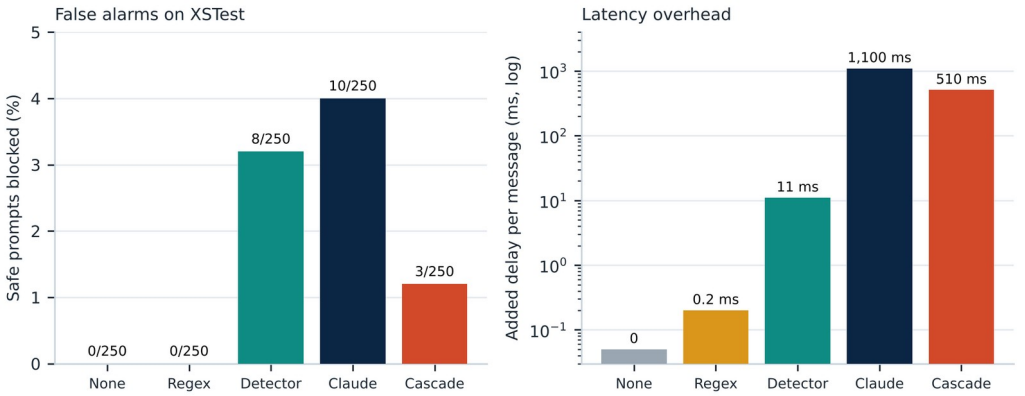


Figure 12. Cost and latency.

10. Responsible AI and safety

- All attacks target a fake shop bot with fake secrets; no real system, person or data is attacked.
- The public demo disables scanning, agent mode and Claude guards, rate-limits by IP (30 per minute) and caps request size at 20 KB.
- API keys live in a git-ignored .env, never in the repo; a spending cap is enforced in code.
- Public datasets are used with their licences: XSTest and BBQ (CC BY 4.0), AdvBench and jailbreak_llms (MIT), TruthfulQA (Apache 2.0).
- The attack content is meant for defenders; results report rates and methods, not new exploits.

11. Limitations and failure cases

- Scope: 4 of the OWASP LLM Top 10. Toxicity, bias and hallucination are tested only on small public samples (100, 90 and 60 items). Bias is scored on the answer letter, and truthfulness as a two-choice question, so it is a proxy for hallucination, not free text. Harmful-request compliance is judged by refusal phrases.
- Mostly small local models (3B) as targets; larger models may behave differently.
- Attack sets are written by us and Claude, not collected in the wild.
- Zero is not safety: 0 of 509 only bounds the true rate below about 0.7% (Wilson upper bound).
- The detector flags 16% of harmless requests; that is why the cascade exists.
- Judge labels came from one person; the final agreement figure is optimistic.
- On indirect injection the detector was trained on similar wording, so its zero is probably optimistic and it removes 25 to 38 percent of harmless documents. The second judge agrees only moderately (kappa 0.47).
- Single-language focus (English plus some Hinglish).
- A v2 detector was trained and rejected: it did not beat the current one on clean data.

12. Mapping to typical judging criteria

Criterion	Evidence
Innovation	Cost-of-safety measurement, cascade, indirect-injection defences, adaptive attackers, judge validation
Technical depth	Trained detector, statistical tests, OpenAI-compatible gateway, agent policy in code
Completeness	Attack generation, guards, judges, metrics, dashboard, service, deployment
Rigour	Clean held-out sets, overlap filter, confidence intervals, judge kappa, public benchmarks
Usability	Dashboard, live console, one-command setup, Docker, public demo
Honesty	Limits section, stated scope, optimistic numbers flagged
Engineering	90 tests, CI, regression gate, versioned releases

13. Run it yourself

Setup (Windows PowerShell, Python 3.11+):

```
git clone https://github.com/tonymadhuri2088-cmd/shieldbench
cd shieldbench
python -m venv .venv ; .venv\Scripts\activate
pip install -r requirements.txt
pytest -q                        # 90 tests
streamlit run app/dashboard.py   # dashboard
python -m shieldbench.live       # service
```

Repository: https://github.com/tonymadhuri2088-cmd/shieldbench

Results page: https://tonymadhuri2088-cmd.github.io/shieldbench/

Live demo: https://shieldbench-demo.onrender.com

14. Future work

- Free-text hallucination and toxicity scoring with a judge model, and larger public samples.

- Multi-turn agent benchmark with tool-using attackers.
- Test larger and commercial targets with the second-family judge.
- Collect real-world injected documents instead of templates.
- Per-user dashboards and a daily spending cap in the hosted service.